

Tab 1

Day 14 — DNS, MX, TXT, SPF, DKIM, DMARC

Mail & Email-Security Posture Assessment for 3 Organizations

Method: Live DNS queries (equivalent to `dig/lookup`) against public resolvers, run directly against each domain's MX, root TXT, `_dmarc.<domain>` TXT, and (where identifiable) DKIM selector records. No third-party scanner (MXToolbox etc.) was used — records were pulled and interpreted manually so the policy logic is shown, not just a pass/fail badge.

Targets: Zerodha (`zerodha.com`), State Bank of India (`sbi.co.in`), Juspay (`juspay.in`) — three Indian organizations spanning retail fintech, public-sector banking, and B2B payments infrastructure, chosen so the comparison covers Google Workspace and Microsoft 365 as mail providers and a spread of SPF/DMARC policy strength.

1. What each record type reveals

Record	Reveals
MX	Who actually handles the org's inbound mail (Google Workspace, Microsoft 365, self-hosted, a regional provider) — tells you which platform an attacker/investigator should target for phishing-kit realism or account-recovery flows
TXT (root)	Every third-party service the domain has verified ownership for (Google, Microsoft, GlobalSign, Zoho, Apple, marketing tools) — root TXT often leaks the org's SaaS stack before anything else does
SPF (a TXT record, <code>v=spf1 ...</code>)	Which sending IPs/services are authorized to send mail as the domain, and what should happen if a message doesn't match (<code>-all</code> hard fail, <code>~all</code> soft fail, <code>?all</code> neutral)
DKIM	Whether outbound mail is cryptographically signed so receiving servers can verify it wasn't altered/spoofed in transit
DMARC	The enforcement layer that ties SPF+DKIM together and tells receiving mail servers what to do with mail that fails both: <code>none</code> (monitor only), <code>quarantine</code> (spam folder), <code>reject</code> (bounce outright)

2. Zerodha (zerodha.com)

Mail provider: Google Workspace ([aspmx.l.google.com](#) + [googlemail.com](#) MX cluster, standard 5-host Google MX pattern).

SPF:

v=spf1 include:_spf.google.com ip4:180.179.153.155 ip4:124.153.66.254 ip4:180.179.153.181 ip4:35.154.75.169 -all

- Authorizes Google Workspace plus four explicit IPv4 addresses (likely internal application/transactional mail servers — order confirmations, trade alerts).
- Ends in **-all (hard fail)** — the strongest SPF policy. Anything not on this list is explicitly rejected by receiving servers that check SPF.

DKIM: Configured. [google._domainkey.zerodha.com](#) returns a live 2048-bit RSA public key, confirming Google Workspace DKIM signing is active on outbound mail.

DMARC:

v=DMARC1; p=quarantine; sp=quarantine; pct=100; rua=mailto:dmarc@zerodha.com

- Policy: **quarantine**, applied to 100% of mail (**pct=100**), and explicitly extended to subdomains (**sp=quarantine**).
- Aggregate reports (**rua**) go to an internal mailbox, so the security team is receiving visibility into failures — but no forensic reports (**ruf**) and no **fo=** tag.

Assessment: Strong SPF (hard fail) + working DKIM + enforced DMARC at quarantine. This is a mature, well-run configuration — spoofed mail claiming to be from Zerodha would be flagged and typically routed to spam by any DMARC-respecting receiver. The one gap relative to "best possible" is that DMARC stops at **quarantine** rather than **reject**, so a spoofed message can still land in some inboxes (just marked as spam) rather than being bounced outright.

3. State Bank of India (sbi.co.in)

Mail provider: Microsoft 365 ([sbi-co-in.mail.protection.outlook.com](#)).

SPF:

v=spf1 ip4:175.158.69.0/19 ip4:175.158.69.32/29 ip4:175.158.67.62/29 ip4:202.162.236.62/29 include:spf.protection.outlook.com -all

- Combines explicit internal IP ranges (SBI's own core-banking/notification infrastructure) with the standard Microsoft 365 include.
- Also ends in **-all (hard fail)**.

Root TXT also shows: `globalsign-domain-verification`, plus several bare verification tokens — indicating GlobalSign is used for domain-validated certs/identity, on top of the Microsoft 365 stack.

DKIM: Both default Microsoft selectors are live — `selector1._domainkey.sbi.co.in` and `selector2._domainkey.sbi.co.in` resolve as CNAMEs into `sbionline.onmicrosoft.com`, which is exactly how Microsoft 365 DKIM signing is delegated for a custom domain. This confirms DKIM is enabled, not just SPF.

DMARC:

v=DMARC1;p=reject;rua=mailto:dmarc.alerts@sbi.co.in;ruf=mailto:dmarc.ruf@sbi.co.in;fo=1

- Policy: **reject** — the strongest possible enforcement. Mail failing SPF+DKIM alignment is bounced, not just spam-folded.
- Both aggregate (`rua`) and forensic (`ruf`) reporting addresses are configured, with `fo=1` (generate a forensic report on any failure, not just full failure) — this is a genuinely well-instrumented setup, consistent with a large regulated bank that has almost certainly been a heavy phishing/spoofing target and has hardened accordingly.

Assessment: This is close to a textbook-ideal configuration — hard-fail SPF, live DKIM on both selectors, and DMARC at `p=reject` with full reporting. Domain spoofing of `sbi.co.in` directly (i.e., "From: alerts@sbi.co.in") is very difficult against any mail provider that enforces DMARC. (This doesn't stop lookalike-domain phishing like `sbi-bank-alert.in` or `sb1.co.in` — DMARC only protects the exact domain it's published on.)

4. Juspay (juspay.in)

Mail provider: Google Workspace (same 5-host MX pattern as Zerodha).

SPF:

v=spf1 include:_spf.google.com include:amazonses.com include:zcsend.net
include:usermail.zohocreator.in include:spf.zoho.com include:one.zoho.in include:transmail.net
~all

- Notably broad: seven separate **include:** mechanisms — Google Workspace, Amazon SES, Zoho Campaigns (**zcsend.net**), Zoho Creator, Zoho Mail, Zoho One, and Transmail (a transactional email service). This is a much larger authorized-sender surface than Zerodha or SBI, consistent with a B2B company running marketing (Zoho Campaigns), transactional/API mail (SES, Transmail), and regular Zoho-hosted mail simultaneously.
- Ends in **~all (soft fail)**, not **-all**. Mail from an unlisted source is marked as suspicious/likely-fail but **not** rejected — receiving servers are told to accept-but-flag rather than block.

DKIM: No record at the default **google._domainkey.juspay.in** selector — this doesn't mean DKIM is absent, but it does mean it isn't verifiable via the common selector name; Juspay's various senders (Zoho, SES, Transmail) each use their own provider-specific selectors that aren't guessable without access to a real outbound message header. This is a genuine limitation of black-box DKIM recon: you generally need a sample email to read the **DKIM-Signature** header and learn the actual selector in use.

DMARC:

v=DMARC1; p=quarantine; rua=mailto:dmarc@juspay.in

- Policy: **quarantine**, but no **pct=** (defaults to 100%), no **sp=** (subdomain policy inherits the same value, but isn't explicit), and only aggregate reporting — no forensic reports.

Assessment: This is the weakest of the three, and it's a useful case study precisely because the weakness is about *breadth*, not neglect. Seven authorized senders plus a soft-fail SPF plus quarantine-only DMARC means: (a) the attack surface for SPF include-chain compromise is larger — if any one of those seven services is compromised or misconfigured, mail "from" juspay.in could pass SPF; (b) **~all** explicitly tells receivers not to hard-reject unlisted senders, so a spoofed message from an IP not on any of those seven lists is still accepted and merely flagged; (c) DMARC quarantine means it lands in spam, not bounced — a target org whose spam filtering is lax could still see the spoofed mail. For an org handling **payments infrastructure**, this combination is worth flagging: a convincing phishing email impersonating Juspay to a merchant or partner has a meaningfully higher chance of reaching an inbox than the same attempt against Zerodha or SBI.

5. Comparative summary

	Zerodha	SBI	Juspay
Mail provider	Google Workspace	Microsoft 365	Google Workspace
SPF mechanism count	1 include + 4 IPs	4 IP ranges + 1 include	7 includes
SPF policy	-all (hard fail)	-all (hard fail)	~all (soft fail)
DKIM confirmed	Yes (Google selector, live key)	Yes (both MS selectors, CNAME-delegated)	Not verifiable at default selector
DMARC policy	quarantine, pct=100, sp=quarantine	reject, fo=1	quarantine
DMARC reporting	Aggregate only	Aggregate + forensic	Aggregate only
Overall posture	Strong	Strongest	Weakest of the three

Ranking (strongest → weakest enforcement): SBI > Zerodha > Juspay.

6. What a weak configuration means in practice

- **No DMARC record at all** (not seen in these three, but common elsewhere): SPF and DKIM can both exist and still do nothing to stop spoofing, because nothing tells the receiving server what to *do* about a failure — an attacker can send "From: ceo@company.com" and many mail clients will display it as legitimate.
- **DMARC p=none**: the org gets visibility (aggregate reports) but spoofed mail is delivered exactly as if it passed. This is normally a deliberate, temporary rollout stage before moving to quarantine/reject.
- **~all or ?all SPF**: unlisted senders aren't blocked, just flagged — the actual blocking decision is deferred to whatever DMARC policy is in place (or to the receiving mail server's own heuristics if DMARC is weak or absent too).
- **Missing/unverifiable DKIM**: even with strict SPF+DMARC, without DKIM a message forwarded through an intermediate server (which breaks SPF alignment) has no independent cryptographic proof of authenticity — DKIM is what survives forwarding, SPF often doesn't.

The practical takeaway for OSINT/threat-recon purposes: MX tells you *who* to imitate infrastructure-wise, SPF's include list tells you *how many* legitimate paths exist (and therefore

how many supply-chain weak points), and DMARC's policy value tells you *how far a spoofed message will actually get* before something stops it.

Note on methodology: a handful of larger domains initially considered (Amazon, Canva, Freshworks, and several major Indian consumer apps) returned TXT record sets too large for a truncation-safe UDP query in this environment and were excluded in favor of domains where the full record set could be retrieved and verified directly rather than partially reconstructed.

Tab 2

Day 15 – Hosting, ASN, CDN, SSL Infrastructure Report

Topic: Identifying who actually hosts a website, ASN/IP ownership, CDN detection, SSL certificate inspection **Targets:** GitHub (github.com), Stripe (stripe.com), Shopify (shopify.com)
Date: 2026-07-22

1. Methodology

For each target, the following passive OSINT steps are used:

1. **Resolve the domain** to its current A/AAAA records (`dig`, `nslookup`, or `host`).
2. **IP → ASN lookup** via routing intelligence sources (bgp.he.net, bgpview.io, bgp.tools, RIPEstat, IPinfo) to identify the announcing Autonomous System and its registered owner.
3. **CDN detection** via:
 - Response headers (`server`, `cf-ray`, `cf-cache-status`, `x-served-by`, `x-cache`, `via`, `x-fastly-request-id`, `akamai-*`)
 - ASN of the resolved IP (CDN/edge ASNs are well-known — e.g. AS13335 = Cloudflare, AS54113 = Fastly, AS16625/AS20940 = Akamai)
 - Public case studies / vendor disclosures confirming CDN usage
4. **SSL/TLS certificate inspection** via:
 - `openssl s_client -connect host:443 -servername host` for a live handshake (issuer, validity window, SANs, chain)
 - Certificate Transparency logs (crt.sh, Censys) for historical/parallel issuance records

Note on data freshness: ASN ownership is generally stable, but a website's **CDN edge IP, exact SSL certificate serial number, and validity dates rotate frequently** (Let's Encrypt certs reissue every ~90 days; enterprise CDNs rotate certs on their own schedule). The figures below reflect the infrastructure model confirmed via public documentation/case studies at the time of writing. **Before relying on cert validity dates or specific IPs for an assessment, re-run the live commands in §5 against the current state.**

2. Target 1: GitHub (github.com)

Attribute	Finding
Hosting model	Primarily self-hosted (GitHub operates its own data center / colocation footprint) rather than renting from a third-party cloud, with Fastly in front for edge delivery.
ASN (origin)	AS36459 — "GITHUB" (GitHub, Inc.), registered with ARIN. Roughly 24 IPv4 / 2 IPv6 prefixes originated, ~9,200 IPv4 addresses announced.
ASN owner	GitHub, Inc. (self-operated network, not a hosting reseller)
CDN presence	Yes — Fastly. Confirmed via Fastly's own published customer case study: Fastly sits in front of github.com, GitHub Pages, and raw.github.com, handling static asset delivery and absorbing DDoS traffic with "grace mode" caching.
CDN detection tell-tales	Look for x-served-by , x-cache , x-fastly-request-id response headers, and Via: 1.1 varnish — classic Fastly/Varnish signatures.
SSL/TLS cert	Issued by DigiCert (GitHub's historical CA — DigiCert TLS RSA/ECC CAs) or Sectigo , depending on cert generation; multiple SANs typically covering github.com , www.github.com , and related first-party domains. Run the live check in §5 to get current issuer, serial, NotBefore/NotAfter, and full SAN list — GitHub has migrated CAs before and rotates certs on a normal enterprise cadence.

Assessment: GitHub is a useful "hybrid" example — the ASN is *not* a generic cloud/hosting provider (they run their own network, AS36459), but the content-delivery layer is outsourced to Fastly. This is a common pattern to recognize: **ASN ownership ≠ who terminates your TLS connection or serves your static content.**

3. Target 2: Stripe (stripe.com)

Attribute	Finding
Hosting model	Multi-provider: public-facing marketing/docs site fronted by a CDN; backend/API and webhook-originating infrastructure documented as running on AWS .

ASN (edge, as observed)	AS13335 — Cloudflare, Inc. (San Francisco, CA) when resolving the public website — i.e., the IP a client connects to belongs to Cloudflare's edge network, not Stripe directly.
ASN (backend, per Stripe's own docs)	AS16509 — AMAZON-02 (Amazon.com, Inc. / AWS) — Stripe publishes IP ranges for webhooks and API traffic that fall within Amazon's ASN, confirming AWS as (at least part of) their compute backend.
CDN presence	Yes — Cloudflare. The public site resolves into Cloudflare's anycast IP space (AS13335); actual origin infrastructure is hidden behind it.
CDN detection tell-tales	<code>cf-ray</code> , <code>cf-cache-status</code> , <code>server: cloudflare</code> headers; SSL cert issued to a Cloudflare-managed CA bundle if using Cloudflare's universal SSL, or a Stripe-owned cert if using Cloudflare in "full (strict)" mode with origin cert pinning.
SSL/TLS cert	If terminated at Cloudflare edge: issuer typically Google Trust Services or Let's Encrypt (Cloudflare's default universal SSL CAs), SANs often include a wildcard plus the apex domain. If Stripe pins its own cert: issuer would be Stripe's chosen enterprise CA (historically DigiCert). Distinguishing which mode is in effect requires the live handshake in §5 — Cloudflare-fronted sites can present either their own default cert or the origin's cert depending on configuration, and this is a key thing to check by hand rather than assume.

Assessment: Stripe illustrates the "ASN you see is not the ASN that matters" problem directly — a naive IP→ASN lookup on stripe.com returns Cloudflare, which tells you nothing about where Stripe's actual servers live. Only Stripe's own published IP-range documentation (used for webhook allowlisting) reveals the AWS/AS16509 backend footprint.

4. Target 3: Shopify (shopify.com / *.myshopify.com storefronts)

Attribute	Finding
Hosting model	Historically multi-cloud (Shopify has previously named Google Cloud Platform and, earlier, Fastly, in public engineering commentary); currently, CDN/edge delivery for merchant storefronts is explicitly documented as Cloudflare-backed .

ASN (edge, as observed)	AS13335 — Cloudflare, Inc. Shopify's own developer documentation states plainly that it provides merchants a CDN "backed by Cloudflare," with assets served via <code>cdn.shopify.com</code> using HTTP/3 and TLS 1.3.
CDN presence	Yes — Cloudflare , confirmed directly by Shopify (both in developer docs and in a published Cloudflare customer case study describing Shopify's use of Cloudflare Workers for checkout logic).
CDN detection tell-tales	<code>cf-ray</code> , <code>server: cloudflare</code> on both the main site and merchant storefronts; asset URLs routed through <code>cdn.shopify.com</code> .
SSL/TLS cert	Storefront certs are typically Cloudflare-managed (auto-issued, often Google Trust Services or Let's Encrypt , wildcard-style SAN sets covering many merchant subdomains per cert in some multi-domain SNI configurations). Shopify's own apex domain (<code>shopify.com</code>) cert issuer should be confirmed live — enterprise sites of this size often use DigiCert or Sectigo for the primary corporate domain even when merchant storefronts ride on Cloudflare's shared cert infrastructure.

Assessment: Shopify is the clearest "pure CDN-fronted SaaS" example of the three — both the platform's own admissions and third-party case studies agree Cloudflare sits in front of essentially all customer-facing traffic, making origin-server identification the hard part (deliberately, since Shopify hosts millions of tenants behind the same edge).

5. Live Verification Commands (run these to get current, exact data)

--- DNS resolution ---

`dig +short github.com`

`dig +short stripe.com`

`dig +short shopify.com`

--- ASN lookup for a resolved IP (replace <IP>) ---

`whois -h whois.cymru.com " -v <IP>"`

`curl -s "https://stat.ripe.net/data/prefix-overview/data.json?resource=<IP>"`

--- CDN detection via headers ---

`curl -sI https://github.com | grep -iE "server|cf-ray|x-served-by|x-cache|via"`

`curl -sI https://stripe.com | grep -iE "server|cf-ray|x-served-by|x-cache|via"`

`curl -sI https://shopify.com | grep -iE "server|cf-ray|x-served-by|x-cache|via"`

```
# --- Live SSL certificate inspection ---
echo | openssl s_client -connect github.com:443 -servername github.com 2>/dev/null | openssl
x509 -noout -issuer -dates -subject -ext subjectAltName
echo | openssl s_client -connect stripe.com:443 -servername stripe.com 2>/dev/null | openssl
x509 -noout -issuer -dates -subject -ext subjectAltName
echo | openssl s_client -connect shopify.com:443 -servername shopify.com 2>/dev/null | openssl
x509 -noout -issuer -dates -subject -ext subjectAltName

# --- Certificate Transparency history (crt.sh) ---
# https://crt.sh/?q=github.com
# https://crt.sh/?q=stripe.com
# https://crt.sh/?q=shopify.com
```

6. Key Takeaways

1. **ASN ≠ hosting provider ≠ CDN.** All three can be different entities, and often are for large sites. GitHub runs its own ASN but outsources edge delivery to Fastly. Stripe and Shopify both sit behind Cloudflare at the edge while backend compute lives elsewhere (AWS, GCP).
2. **Public documentation is often the fastest recon source.** Vendor case studies (Fastly's GitHub case study, Cloudflare's Shopify case study, Stripe's own published IP-range docs) frequently confirm infrastructure relationships faster and more reliably than inferring from headers alone.
3. **A CDN in front of a target hides the true origin ASN/IP** — this is intentional (DDoS mitigation, load distribution) and means IP/ASN lookups on the public hostname will return the CDN's network, not the organization's own infrastructure. Origin discovery (historical DNS records, SSL cert SANs on non-CDN subdomains, leaked origin IPs) is a separate, more involved technique.
4. **SSL certificate details are the most time-sensitive data point in this report** — always re-verify issuer, validity window, and SAN list live rather than relying on a point-in-time report, since certs rotate on 90-day (Let's Encrypt) to multi-year (enterprise CA) cycles.

Tab 3

Day 16 – Passive DNS & Subdomain Enumeration Report

Topic: Passive DNS databases, non-intrusive subdomain enumeration **Target:** GitHub (github.com) — continued from Day 15 for infrastructure consistency **Date:** 2026-07-22

1. Methodology & Tooling Notes

Passive subdomain enumeration pulls subdomains from data sources that **never touch the target's servers** — no active DNS brute-forcing, no port scanning. Sources used/recommended for this exercise:

Source	What it gives you	Access note
crt.sh (Certificate Transparency logs)	Every hostname ever put in a SAN field of a publicly-issued cert for the domain	crt.sh's web UI blocks automated/scripted requests (robots.txt disallow) — query it manually in a browser, or use the JSON API from a normal client with a User-Agent, not a bot. See §5 for the exact query.
DNSDumpster	Aggregates passive DNS + crt.sh + reverse-IP data with a nice visual map	Manual/browser use recommended (also rate-limits automated queries)
Sublist3r-style tools	Combine crt.sh, VirusTotal, ThreatCrowd, Google/Bing dorking, DNS records into one enumeration pass	Run locally; several data sources it depends on rate-limit or block cloud/datacenter IPs
GitHub's own published API/docs	GitHub, being a dev-facing company, documents many of its own subdomains directly (API root, meta endpoints, asset CDNs) — a legitimate, zero-request source of ground truth	No blocking, first-party

Important limitation of this report: the subdomain list below was compiled from GitHub's own public documentation, its published API metadata, and well-known/publicly-documented infrastructure hostnames — **not from a live crt.sh scrape**, since that requires either a browser session or a non-automated client (blocked from this environment). Treat this as a solid *starting* map; run the commands in §5 yourself for a complete, current, exhaustive list, since Certificate Transparency logs for a domain this size will contain many more entries (marketing landing pages, regional docs mirrors, old decommissioned test certs, etc.) than any curated list can capture by hand.

2. Subdomain Inventory (github.com)

Subdomain	Purpose
github.com	Apex / main web application
www.github.com	WWW redirect
api.github.com	REST/GraphQL API
gist.github.com	Gist code-snippet sharing service
raw.githubusercontent.com	Raw file content delivery (separate domain, same org)
codeload.github.com	Repository archive/tarball downloads
objects.githubusercontent.com	Git LFS / release asset object storage
avatars.githubusercontent.com	User/org avatar image CDN
camo.githubusercontent.com	Image proxy (camouflage proxy for mixed-content images in READMEs)
media.githubusercontent.com	Media asset delivery
github.githubassets.com	Static front-end assets (CSS/JS bundles)
docs.github.com	Documentation portal
support.github.com	Support ticketing/help center
education.github.com	GitHub Education / Student Developer Pack

<code>skills.github.com</code>	GitHub Skills learning platform
<code>desktop.github.com</code>	GitHub Desktop app download/update site
<code>cli.github.com</code>	GitHub CLI (<code>gh</code>) download/docs
<code>classroom.github.com</code>	GitHub Classroom
<code>pages.github.com</code>	GitHub Pages marketing/info page
<code>githubusercontent.com</code> (wildcard tenant domain, e.g. <code>*.github.io</code>)	User/org static site hosting (Pages)
<code>githubstatus.com</code>	Status/incident page (separate root domain, same org — common pattern)
<code>github.blog</code>	Engineering/company blog (separate root domain)
<code>githubcopilot.com</code>	Copilot product site (separate root domain)
<code>central.github.com</code>	SSO / enterprise identity endpoint (referenced in GitHub Enterprise docs)
<code>uploads.github.com</code>	File upload handling endpoint (referenced in API docs/CSP headers)

3. Categorization

Public-facing, expected (production/customer-facing)

- `github.com`, `www.github.com`, `api.github.com`, `gist.github.com`, `raw.githubusercontent.com`, `codeload.github.com`, `docs.github.com`, `support.github.com`, `education.github.com`, `skills.github.com`, `desktop.github.com`, `cli.github.com`, `classroom.github.com`, `pages.github.com`, `githubstatus.com`, `github.blog`, `githubcopilot.com`

These are all intentionally public products or marketing/support surfaces — nothing unusual here.

CDN / asset infrastructure (public by design, but not "web app" surface)

- `objects.githubusercontent.com`, `avatars.githubusercontent.com`, `camo.githubusercontent.com`, `media.githubusercontent.com`, `github.githubassets.com`, `*.github.io`

These exist specifically to serve static/user content at scale and are meant to be internet-facing — flagging them isn't a finding, but recognizing *why* they're separate domains (cookie/session isolation, so a malicious upload can't read github.com's session cookies) is a good practice to note in a report.

"Looks internal, but isn't" (identity/SSO surface worth a second look)

- `central.github.com` — this is an **authentication/SSO-adjacent endpoint**. It's technically meant to be reachable (enterprise users hit it during login flows), but naming like "central" is exactly the kind of subdomain that would warrant closer manual inspection in a real assessment: confirm it requires auth, isn't leaking a login form with verbose errors, and isn't an old/deprecated identity provider integration point that nobody's monitoring anymore.
- `uploads.github.com` — file upload endpoints are a classic target for testing (SSRF via upload processing, stored content abuse) even when intentionally public.

None found in this pass: dev/staging/internal test subdomains

No `dev.`, `staging.`, `test.`, `internal.`, `qa.`, `beta.`, `vpn.`, or `admin.` -style subdomains turned up in this manually-compiled pass. **This is very likely a gap in the methodology, not a gap in GitHub's actual DNS/cert footprint** — companies at this scale almost always have *some* certs issued for internal or staging hostnames that leak into CT logs (a staging environment behind a WAF still needs a publicly-trusted cert if it's ever hit from outside a private network). A live crt.sh query (§5) with wildcard search (`%.github.com`) is very likely to surface these, and they are exactly the category to prioritize once found — see §4.

4. What Shouldn't Be Public-Facing (Findings Prioritization)

If a live crt.sh/DNSDumpster pass on any target turns up subdomains matching these patterns, prioritize them for manual review — they're the highest-value findings in passive subdomain recon:

Pattern

Why it matters

<code>dev.*</code> , <code>staging.*</code> , <code>test.*</code> , <code>qa.*</code> , <code>uat.*</code>	Often run with debug mode on, verbose error pages, weaker auth, or outdated software versions compared to production
<code>internal.*</code> , <code>corp.*</code> , <code>intranet.*</code> , <code>vpn.*</code>	Should almost never resolve publicly or should require VPN/allowlisted IP — if it resolves and responds, that's a real finding
<code>admin.*</code> , <code>panel.*</code> , <code>dashboard.*</code> , <code>cpanel.*</code>	Admin surfaces are high-value targets if exposed without strong auth in front
<code>api-dev.*</code> , <code>api-staging.*</code> , <code>sandbox.*</code>	Frequently have relaxed rate limits, test data, or auth bypasses left in from development
<code>old.*</code> , <code>legacy.*</code> , <code>backup.*</code> , <code>*-bak.*</code>	Decommissioned-but-still-resolving infrastructure — often unpatched and unmonitored
Anything with a cert issued once, years ago, never renewed	A single old CT log entry with no follow-up cert usually means an abandoned host — worth checking if it still resolves/responds

For GitHub specifically: nothing found in this manual pass falls into these categories, which is consistent with GitHub being a security-mature engineering organization that keeps non-production infrastructure off public DNS/CT logs wherever possible (internal environments typically use internal-only CAs or private DNS zones that never touch public Certificate Transparency logs at all — itself a good practice to note when writing up a target that does this well).

5. Commands to Run for a Live, Complete Enumeration

```
# --- crt.sh wildcard query (browser or curl with a normal UA, not automated bot traffic) ---
curl -s -A "Mozilla/5.0" "https://crt.sh/?q=%.github.com&output=json" | jq -r '.[].name_value' | sort -u
```

```
# --- Sublist3r (aggregates multiple passive sources) ---
python3 sublist3r.py -d github.com -o github_subdomains.txt
```

```
# --- Amass (passive mode only, no active brute force) ---
amass enum -passive -d github.com -o amass_github.txt
```

```
# --- DNSDumpster (manual browser use recommended) ---
```

```
# https://dnsdumpster.com/ -> search "github.com"
```

```
# --- Cross-check which discovered subdomains actually resolve ---  
cat github_subdomains.txt | while read sub; do  
    ip=$(dig +short "$sub" | tail -1)  
    [ -n "$ip" ] && echo "$sub -> $ip"  
done
```

6. Key Takeaways

1. **Certificate Transparency is the single richest passive source** for subdomain discovery — any publicly-trusted cert issuance is permanently logged and searchable, which is exactly why an organization's staging/internal hosts leak if they're ever issued a public CA cert.
2. **crt.sh actively blocks scripted/bot access** at the web-UI level — a real assessment needs either a proper API client with reasonable rate limiting and a normal User-Agent, or a local tool (Sublist3r, Amass, OWASP Amass, etc.) that handles this correctly.
3. **The absence of dev/staging/internal subdomains in a hand-compiled pass is not the same as their absence in reality** — always run the live wildcard query before concluding a target has no exposed non-production infrastructure.
4. **Separate CDN/content domains (e.g. githubusercontent.com vs github.com) are usually a deliberate security boundary** (cookie/session isolation for user-generated content), not a misconfiguration — worth distinguishing in a report from genuine "shouldn't be public" findings like central.github.com or uploads.github.com, which merit a closer look even though they're intentionally reachable.